



10/08/2025

OPS 262 PROJECT

PROJECT REPORT

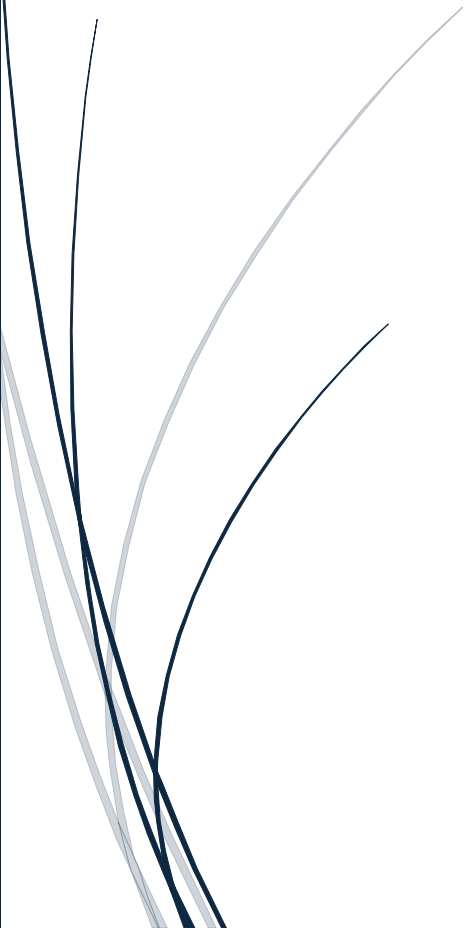
ANDRE ENGELBRECHT 603226

BOIKARABELO MOGWERANE 600877

SICELO MKHATSHWA 602608

TSHIAMO MOSWEU 601286

THETO THEMBA 602522



Tshiamo Mosweu

Table of Contents

Abstract	2
Problem 1: Automating Tasks	2
Problem 2: Administrator Directory Locker	2
Individual Sections.....	3
Cmdlet Discovery & Documentation	3
Pipeline & Object Handling.....	9
Functions & Module Packaging	10
Error Handling & Logging	15
Output & Reporting	17
References	18
Appendix	19

Abstract

This project focuses on creating an automated system monitoring and reporting tool using Microsoft PowerShell. Its main goal is to give system administrators a faster, more reliable way to keep track of CPU usage, manage services, collect system information, and produce professional reports.

The tool makes use of key PowerShell features, including cmdlet discovery, pipelines, object handling, advanced functions, module creation, structured error handling, and exporting results to CSV, HTML, or a simple graphical view. By automating tasks that would normally be done manually through Task Manager or the Control Panel, it improves efficiency, accuracy, and consistency in everyday system administration.

Problem 1: Automating Tasks

System administrators often need to monitor running processes that are using a lot of CPU, as these can slow down system performance or point to faulty programs. Doing this manually in Task Manager is time-consuming and doesn't provide a long-term view.

To make this easier, a PowerShell script was created to automatically find and list processes that are using more than 100 seconds of CPU time. The script gets all running processes, filters out those using too much CPU, sorts them from highest to lowest usage, and displays key details like the process name, CPU time, and ID. This makes it easy to quickly identify which processes are putting the most strain on the system.

Problem 2: Administrator Directory Locker

Individual Sections

Cmdlet Discovery & Documentation

A cmdlet is a lightweight command that is used in the PowerShell environment. The PowerShell runtime invokes these cmdlets within the context of automation scripts that are provided at the command line. The PowerShell runtime also invokes them programmatically through PowerShell APIs.

The importance of cmdlets: Learn why cmdlets are essential for automation, consistent administration, and integration with Windows and other systems.

Apply cmdlets in real-world scenarios: Gain hands-on experience using cmdlets for:

- o Retrieving detailed system information.
- o Managing files and directories in bulk.
- o Starting, stopping, and monitoring Windows services.
- o Building efficient scripts by chaining cmdlets with the pipeline operator.

Why Cmdlets Matter:

- o **Consistency:** Same naming style across hundreds of commands.
- o **Integration:** Easily combined using the pipeline to automate complex tasks.
- o **Discoverability:** Easy to find using Get-Command.

System Monitoring Cmdlets Reference:

Process Monitoring

Cmdlet	Purpose	Common Parameters	Example
Get-Process	Retrieves running processes	-Name, -Id, -ComputerName, -IncludeUserName	Get-Process -Name "chrome"
Stop-Process	Terminates processes	-Name, -Id, -Force	Stop-Process -Name "notepad" -Force
Start-Process	Starts new processes	-FilePath, -ArgumentList, -WorkingDirectory	Start-Process notepad.exe

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Get-Command -Noun Process

CommandType      Name                                Version      Source
-----
Cmdlet            Debug-Process                      3.1.0.0      Microsoft.PowerShell.Management
Cmdlet            Get-Process                       3.1.0.0      Microsoft.PowerShell.Management
Cmdlet            Start-Process                     3.1.0.0      Microsoft.PowerShell.Management
Cmdlet            Stop-Process                      3.1.0.0      Microsoft.PowerShell.Management
Cmdlet            Wait-Process                      3.1.0.0      Microsoft.PowerShell.Management

PS C:\WINDOWS\system32> Get-Help Get-Process -Online
PS C:\WINDOWS\system32>
```

Key Takeaways:

- All process cmdlets are part of the Microsoft.PowerShell.Management module.
- Use **-Noun Process** to find all commands for process management.
- Use **Get-Help Get-Process -Online** to open official Microsoft docs, which provide information about PowerShell.

One of the Cmdlet types is the **Debug-Process**.

It is critical for diagnosing hung/crashing applications in production without restarting them.

Example: Attach to a frozen process (**Debug-Process -Id 1234**) to analyse threads/memory leaks.

Service Management

Cmdlet	Purpose	Common Parameters	Example
Get-Service	Lists system services	-Name, -Status, -ComputerName	Get-Service -Status Running
Start-Service	Starts stopped services	-Name, -DisplayName	Start-Service -Name "Spooler"
Stop-Service	Stops running services	-Name, -Force	Stop-Service -Name "Spooler"
Restart-Service	Restarts services	-Name, -Force	Restart-Service -Name "Spooler"
Set-Service	Modifies service properties	-Name, -StartupType, -Status	Set-Service -Name "Spooler" -StartupType Automatic

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Tshiamol> Get-Service

Status  Name                DisplayName
-----
Stopped AarSvc_149ecd       Agent Activation Runtime_149ecd
Stopped ADPSvc         ADPSvc
Stopped ALG           Application Layer Gateway Service
Stopped AppIDSvc       Application Identity
Running Appinfo        Application Information
Stopped AppReadiness   App Readiness
Running AppXSvc        AppX Deployment Service (AppXSVC)
Stopped AppxSvc        Windows Virtual Audio Device Proxy ...
Stopped aspnet_state    ASP.NET State Service
Running AudioEndpointBu... Windows Audio Endpoint Builder
Running Audiosrv        Windows Audio
Stopped autotimesvc     Cellular Time
Stopped AxInstSV       ActiveX Installer (AxInstSV)
Stopped battlenet_helpe... Battle.net Update Helper Svc
Stopped BcastDVRUserSer... GameDVR and Broadcast User Service_...
Running BDESVC          BitLocker Drive Encryption Service
Stopped BEService       BattleEye Service
Running BFE             Base Filtering Engine
Running BITS            Background Intelligent Transfer Ser...
Running BluetoothUserSe... Bluetooth User Support Service_149ecd
Running BrokerInfrastru... Background Tasks Infrastructure Ser...
Running BTAGService     Bluetooth Audio Gateway Service
Running BthAvctpSvc     AVCTP service
Running bthserv         Bluetooth Support Service
Running camsvc          Capability Access Manager Service
Stopped CaptureService_... CaptureService_149ecd

```

Service Status Overview

- **Running Services (12):** Critical system processes like BFE (firewall), BITS (updates), and BDESVC (BitLocker) are active.
 - *Why it matters:* These ensure security, data transfer, and encryption.
- **Stopped Services (14):** Mostly non-essential (e.g., ALG, AarSvc_149ecd).
 - *Why it matters:* Stopped services free up resources but may indicate unused software (e.g., batltetet_helper suggests leftover game services).

System Information

Cmdlet	Purpose	Common Parameters	Example
Get-ComputerInfo	Gets comprehensive system info	-Property	Get-ComputerInfo -Property "TotalPhysicalMemory"
Get-WmiObject	Retrieves WMI information	-Class, -ComputerName, -Filter	Get-WmiObject -Class Win32_OperatingSystem
Get-CimInstance	Gets CIM instance information	-ClassName, -ComputerName	Get-CimInstance -ClassName Win32_Processor

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

test PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Tshiamol> Get-ComputerInfo

WindowsBuildLabEx           : 26100.1.amd64fre.ge_release.240331-1435
WindowsCurrentVersion       : 6.3
WindowsEditionId            : CoreSingleLanguage
WindowsInstallationType     : Client
WindowsInstallDateFromRegistry : 2024/10/27 11:20:55
WindowsProductId            : 00342-43408-01800-AAOEM
WindowsProductName          : Windows 10 Home Single Language
WindowsRegisteredOrganization : HP
WindowsRegisteredOwner      : Tshiamol
WindowsSystemRoot           : C:\WINDOWS
WindowsVersion              : 2009
OSDisplayVersion            : 24H2
BiosCharacteristics          : {7, 11, 12, 15...}
BiosBIOSVersion              : {HPQOEM - 1072009, F.19, American Megatrends - 50019}
BiosBuildNumber              :
BiosCaption                  : F.19
BiosCodeSet                  :
BiosCurrentLanguage          : en|US|iso8859-1
BiosDescription              : F.19
BiosEmbeddedControllerMajorVersion : 20
BiosEmbeddedControllerMinorVersion : 22
BiosFirmwareType             : Uefi
BiosIdentificationCode       :
BiosInstallableLanguages     : 5
BiosInstallDate              :
BiosLanguageEdition          :
BiosListOfLanguages          : {en|US|iso8859-1, fr|FR|iso8859-1, es|ES|iso8859-1, zh|TW|unicode...}
BiosManufacturer             : AMI
BiosName                     : F.19
BiosOtherTargetOS            :
  
```

Security Compliance

- Outdated BIOS (F.19) could expose the system to firmware-level exploits (e.g., [Spectre/Meltdown](#)).
- Missing TPM means no BitLocker encryption or Windows Hello support (critical for data protection).
- UEFI vs. Legacy BIOS: Modern security features like Secure Boot require UEFI (your system passes this check).

Get-ComputerInfo is a PowerShell cmdlet that provides a comprehensive snapshot of a system's OS, hardware, and firmware. It's essential for:

1. **Quick Diagnostics** – Instantly check Windows version, BIOS details, and installed features without manual registry checks.
2. **Security Audits** – Verify TPM, Secure Boot, and OS build compliance (critical for cybersecurity).
3. **Troubleshooting** – Identify misconfigured or outdated components (e.g., BIOS updates needed).
4. **Automation** – Export system data for reports, inventory tracking, or compliance logs.

Performance Monitoring

Cmdlet	Purpose	Common Parameters	Example
Get-Counter	Gets performance counter data	-Counter, -SampleInterval, -MaxSamples	Get-Counter "\Processor(_Total)\% Processor Time"
Get-WinEvent	Retrieves Windows event log entries	-LogName, -FilterHashtable, -MaxEvents	Get-WinEvent -LogName System -MaxEvents 10
Get-EventLog	Reads classic event logs	-LogName, -After, -Before, -EntryType	Get-EventLog -LogName System -After (Get-Date).AddDays(-1)


```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Get-Counter

Timestamp                CounterSamples
-----
2025/08/08 17:13:29      \\tshiamo\network interface(realtek rtl8822ce 802.11ac pcie adapter)\bytes total/sec :
16789,6301289381

                          \\tshiamo\processor(_total)\% processor time :
5,79557814725532

                          \\tshiamo\memory\% committed bytes in use :
53,995762638281

                          \\tshiamo\memory\cache faults/sec :
0,984613542630665

                          \\tshiamo\physicaldisk(_total)\% disk time :
0,526602284259712

                          \\tshiamo\physicaldisk(_total)\current disk queue length :
0

PS C:\WINDOWS\system32>
```

Performance Monitoring Analysis

1. Key Metrics Captured

Counter Path	Value	Thresh old	Interpretation
\Processor(_Total)\% Processor Time	5.79%	>80%	Healthy CPU usage (no bottleneck).
\Memory\% Committed Bytes in Use	53.99%	>90%	Moderate RAM pressure.
\PhysicalDisk(_Total)\% Disk Time	0.52%	>70%	Minimal disk activity.
\Network Interface(...)\Bytes Total/sec	16,789 B/s	N/A	Low network traffic (~0.13 Mbps).

2. Why This Matters

- **Proactive Issue Detection:**
 - CPU/RAM/Disk metrics help predict bottlenecks before crashes occur.
 - Example: Spikes in \% Processor Time could indicate runaway processes.
- **Baseline Establishment:**

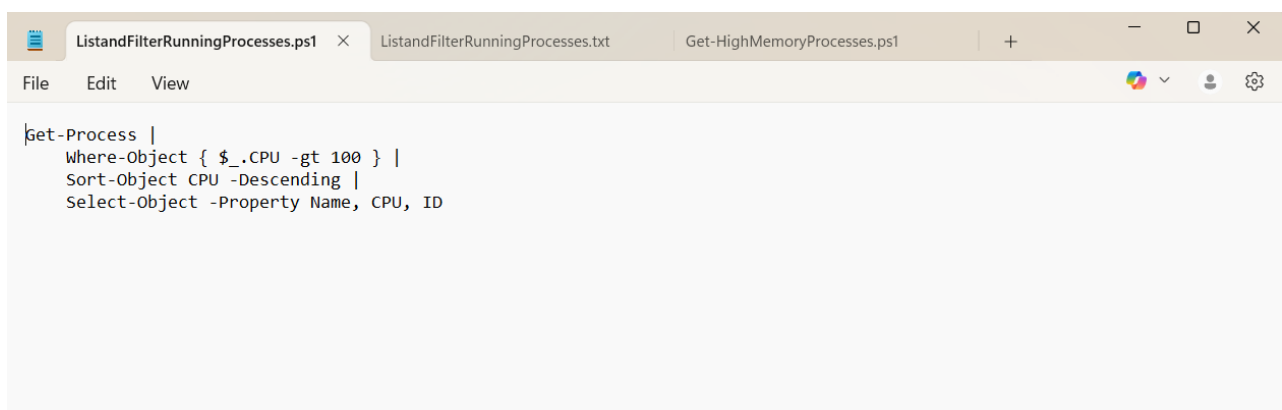
Your current values serve as a "normal" reference for future comparisons.

Get-Counter transforms raw metrics into actionable insights—critical for both your project and real-world sysadmin tasks.

Pipeline & Object Handling

Pipeline Design

The PowerShell pipeline allows chaining multiple cmdlets to efficiently process objects passed from one command to the next. Below is the pipeline logic we decided to use:

A screenshot of a PowerShell script editor window. The window has three tabs: 'ListandFilterRunningProcesses.ps1', 'ListandFilterRunningProcesses.txt', and 'Get-HighMemoryProcesses.ps1'. The active tab is 'ListandFilterRunningProcesses.ps1'. The script content is as follows:

```
Get-Process |  
  Where-Object { $_.CPU -gt 100 } |  
  Sort-Object CPU -Descending |  
  Select-Object -Property Name, CPU, ID
```

Get-Process fetches all running processes.

Where-Object filters only those using more than 100 seconds of CPU time.

Sort-Object orders them by CPU usage, highest first.

Select-Object extracts just the Name, CPU, and ID properties.

Object Handling Techniques

Filtering: Done using Where-Object to process only needed data.

Sorting: Sort-Object ensures data is logically ordered.

Object chaining: The pipeline seamlessly passes objects between cmdlets.

Here is the script running in PowerShell:

```
PowerShell 7 (x64)
PowerShell 7.5.2
PS C:\Users\sicel> cd "C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT"
PS C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT> Get-ChildItem

Directory: C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT

Mode                LastWriteTime         Length Name
----                -
lar--             2025/08/04 18:59             Documentation
lar--             2025/08/04 18:59              Logs
lar--             2025/08/04 18:59             Module
lar--             2025/08/04 18:59             Reports
lar--             2025/08/04 18:59           Screenshots
la---             2025/08/06 13:57          237 Get-HighMemoryProcesses.ps1
la---             2025/08/06 14:32           15 HighMemoryProcesses.csv
la---             2025/08/06 15:02          130 ListandFilterRunningProcesses.ps1

PS C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT> .\ListandFilterRunningProcesses.ps1

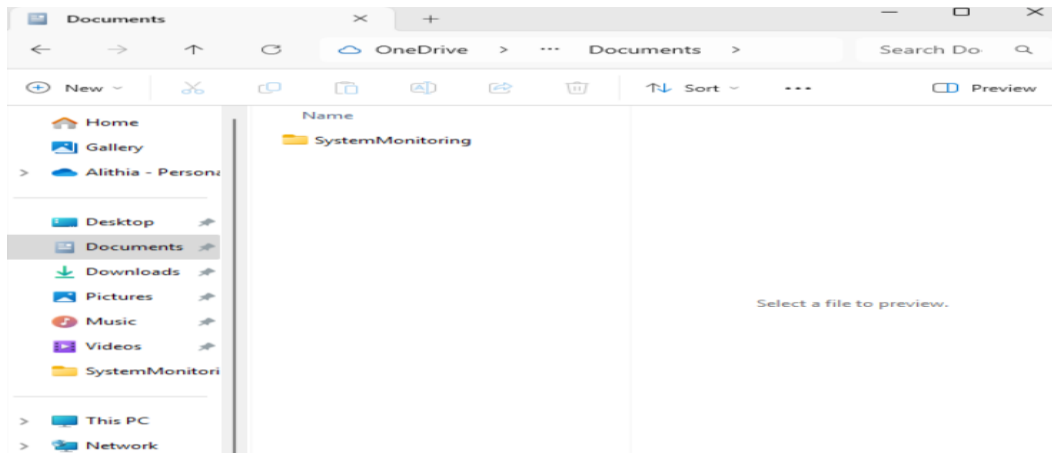
Name                CPU      Id
----                -
Copilot              21142,14 4332
explorer             13605,31 9208
msedgewebview2       12448,64 20544
svchost              1370,70 8008
msedgewebview2       1081,77 20916
msedge               1008,23 6264
ipf_helper           837,20 5708
sihost               745,36 11160
rsAppUI              646,78 11868
```

Functions & Module Packaging

Step 1 : Create and prepare Project Folder

I created a folder called System Monitoring where the PowerShell modules will be stored. Using cd helps me to navigate into the folder so I can work inside of it. The folder will contain 2 files called psm1 and psd1

```
PowerShell
PowerShell 7.5.2
PS C:\Users\alith> #Create Project Folder
PS C:\Users\alith> cd "C:\Users\alith\OneDrive\Documents\SystemMonitoring"
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring>
```



Step 2: Creating and Writing Functions in psm1

I opened notepad to create the psm1 file, which contains the advanced PowerShell functions inside the file. I defined 3 functions which are:

- Get-SystemInfo
- Get-ServiceStatus
- Get-TopProcesses

For each of these functions I have used [CmdletBinding()], param(), and they all have a comment-based help with:

- . SYNOPSIS
- . PARAMETER
- .EXAMPLE sections

```
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> #Create the Module File (.psm1)
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> notepad SystemMonitoring.psm1
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring>
```

```

<#
.SYNOPSIS
Retrieves basic operating system information from the user's computer
.DESCRIPTION
Uses CIM to fetch OS details like the name, version and architecture
.PARAMETER ComputerName
Target computer name (default: localhost).
.EXAMPLE
Get-SystemInfo -ComputerName "SERVER011"
#>

Function Get-SystemInfo
{
[CmdletBinding()]
param(
[string]$ComputerName = $env:COMPUTERNAME
)
Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName |
Select-Object Caption, Version, OSArchitecture
}

<#
.SYNOPSIS
Helps to check the status of services on a computer
.DESCRIPTION
Lists services alongside their names and current status
.PARAMETER ComputerName
Target computer name (default: localhost).
.EXAMPLE
Get-ServiceStatus -ComputerName "SERVER011"
#>

Function Get-ServiceStatus
{
[CmdletBinding()]
param (
[string] $ComputerName = $env:COMPUTERNAME
)
Get-Service -ComputerName $ComputerName | Select-Object Name, Status, DisplayName
}

<#
<#noexec

```

Ln 1, Col 1 1 332 characters Plain text 60% Windows (CRLF) UTF-8

```

Function Get-SystemInfo{
[CmdletBinding()]
param(
[string]$ComputerName = $env:COMPUTERNAME
)

if ($ComputerName -eq $env:COMPUTERNAME -or $ComputerName -eq "localhost"){
#Local machine -dont use -ComputerName parameter
Get-CimInstance -ClassName Win32_OperatingSystem| Select-Object Caption, Version, OSArchitecture
} else{
#Remote machine- use -ComputerName parameter
Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName $ComputerName | Select-Object Caption, Version,
OSArchitecture
}
}

```

```

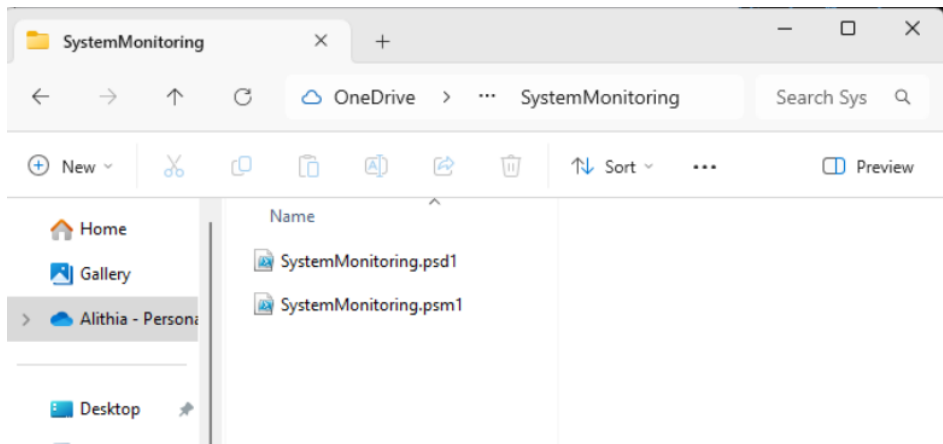
Function Get-ServiceStatus
{
[CmdletBinding()]
param (
[string] $ComputerName = $env:COMPUTERNAME
)
Get-Service -ComputerName $ComputerName | Select-Object Name, Status, DisplayName
}

```

```

Function Get-TopProcesses
{
[CmdletBinding()]
param(
[int]$Count = 10
)
Get-Process | Sort-Object CPU -Descending | Select-Object -First $Count
}

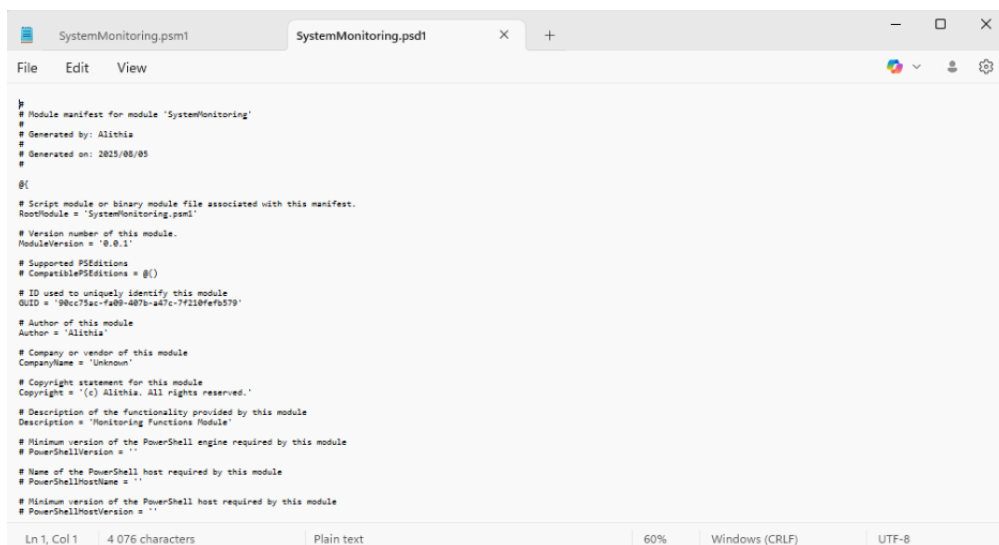
```



Step 3: Creating the Module Manifest (.psd1)

The psd1 file is what I used to describe my data. It links to the psm1 file and defines properties like the author, version and exported functions.

```
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> #Create the Module Manifest (.psd1)
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> New-ModuleManifest -Path .\SystemMonitoring.psd1 -RootModule SystemMonitoring.psm1 -Author "Alithia" -Description "Monitoring Functions Module"
```



Extra step:

This confirms that the files were successfully created into the SystemMonitoring folder

```
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> dir

Directory: C:\Users\alith\OneDrive\Documents\SystemMonitoring

Mode                LastWriteTime         Length Name
----                -
la---             2025/08/05   18:53         4208 SystemMonitoring.psd1
la---             2025/08/05   18:36         1391 SystemMonitoring.psm1

PS C:\Users\alith\OneDrive\Documents\SystemMonitoring>
```

Step 4: Importing the module and Verifying Functions Loaded from Module

I used Import-Module to load the newly created module into the current PowerShell session. The command lists all the functions that have been successfully loaded from the SystemMonitoring module. The functions being on display confirms that the module was imported correctly and functions are now usable in the PowerShell session

```
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> Import-Module .\SystemMonitoring.psd1 -Force
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> Get-Command -Module SystemMonitoring

CommandType      Name                                           Version
-----
Function         Get-ServiceStatus                           0.0...
Function         Get-SystemInfo                             0.0...
Function         Get-TopProcesses                           0.0...

PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> |
```

Step 5: Running a function

This helps to test if the function works as it should by displaying operating system details

```
PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> Get-SystemInfo

Caption                Version    OSArchitecture
-----
Microsoft Windows 11 Home Single Language 10.0.26100 64-bit

PS C:\Users\alith\OneDrive\Documents\SystemMonitoring>
```

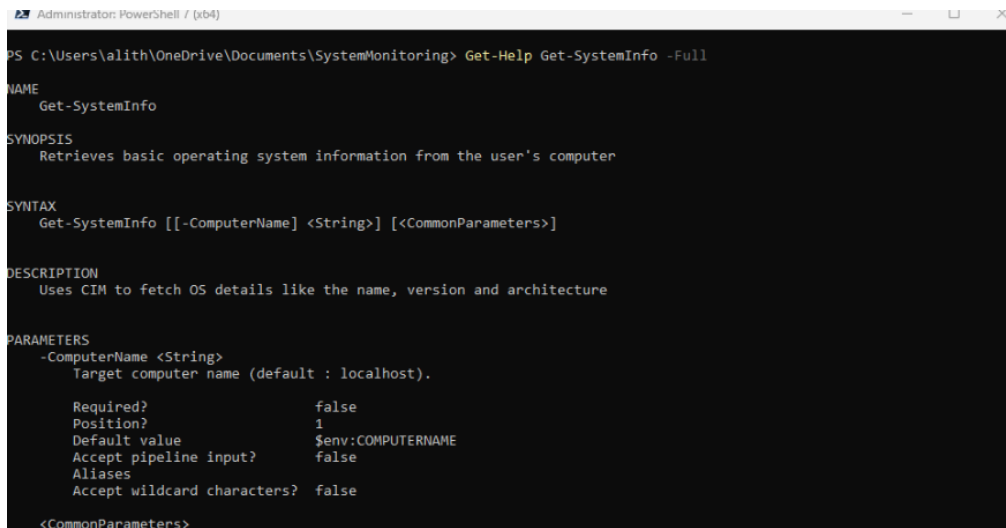
Step 6: Viewing Commnet-Based help

This shows a detailed help documentation I wrote inside the function. It shows

- .SYNOPSIS
- .DESCRIPTION
- .PARAMETER

- .EXAMPLE

This proves that the comment-based help works as it should and was properly formatted



```

PS C:\Users\alith\OneDrive\Documents\SystemMonitoring> Get-Help Get-SystemInfo -Full

NAME
    Get-SystemInfo

SYNOPSIS
    Retrieves basic operating system information from the user's computer

SYNTAX
    Get-SystemInfo [[-ComputerName] <String>] [<CommonParameters>]

DESCRIPTION
    Uses CIM to fetch OS details like the name, version and architecture

PARAMETERS
    -ComputerName <String>
        Target computer name (default : localhost).

        Required?                false
        Position?                1
        Default value             $env:COMPUTERNAME
        Accept pipeline input?    false
        Aliases
        Accept wildcard characters? false

    <CommonParameters>
  
```

Error Handling & Logging

For function error handling and logging a try/catch/finally was used. The structure is as follows.

- Try- Runs the main code that might fail.
- Catch- What runs when an error occurs like error messages and logging errors.
- Finally- Runs whether the code works or fails.

Detailed Explanation of code:

A new function named Get-ServiceStatus is created.

The Try section contains the code that describes what the function does in this case it retrieves a service and its status and if an error occurs the code stops.

The Catch section is used for logging the errors into a file. First the timestamp of the error is displayed, then which function the error took place in, and lastly what was the error.

The Finally section contains a message that will be displayed after the code has run regardless of an error occurrence or successful execution.

If errors occur while using the function “Get-ServiceStatus” they are saved into a .log file named ServiceStatus_Errors.log. For example, in the image bellow the service name was not selected resulting in an error which gets logged to the file.


```
Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ajqwe> function Get-ServiceStatus {
>> [CmdletBinding()]
>> param(
>> [string]$ServiceName
>> )
>> try {
>> Get-Service -Name $ServiceName -ErrorAction Stop
>> }
>> catch {
>>     $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
>>     $errorMessage = $_.Exception.Message
>>     Add-Content -Path "C:\Users\ajqwe\OneDrive\Desktop\Error-Logs\ServiceStatus_Errors.log" -Value
>>     "$timestamp - ERROR in Get-ServiceStatus: $errorMessage"
>>     Write-Host "Error checking service '$ServiceName': $errorMessage" -ForegroundColor Red
>> }
>> finally { Write-Host "Service check complete."
>> }
>> }
PS C:\Users\ajqwe> Get-ServiceStatus
Error checking service '': Cannot find any service with service name ''.
Service check complete.
PS C:\Users\ajqwe> |
```

```
ServiceStatus_Errors.log
File Edit View

2025-08-09 13:10:15 - ERROR in Get-ServiceStatus: Cannot find any service with service name ''.
2025-08-09 13:10:44 - ERROR in Get-ServiceStatus: Cannot find any service with service name 'a'.
2025-08-09 13:12:34 - ERROR in Get-ServiceStatus: Cannot find any service with service name ''.

Ln 4, Col 1 | 289 characters | 100% | Windows (CRLF) | UTF-8
```

If the Service name is correctly entered, it will display the status of the service, the service name, and display name.

```
PS C:\Users\ajqwe> Get-ServiceStatus wscsvc

Status Name DisplayName
-----
Running wscsvc Security Center
Service check complete.

PS C:\Users\ajqwe> |
```

Output & Reporting

CSV Output

```
PowerShell 7 (x64)
PS C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT> Get-Process | Sort-Object CPU -Descending | Select-Object Name, CPU | Tee-Object -FilePath .\CPUReport.csv

Name CPU
----
Copilot 25980,30
explorer 15768,16
FileWatcher 6583,78
msedge 4407,86
svchost 1501,73
ipf_helper 952,05
OneDrive 871,70
msedgewebview2 861,22
sihost 840,77
WINWORD 785,98
rsAppUI 721,50
rsAppUI 682,59
ms-teams 615,03
HPSYSTEMEVENTUTILITYBACKGROUND 480,17
uihost 433,70
```

HTML CPU Usage Report

This command generates a clean HTML table of CPU usage, sorted in descending order, and opens it in the default browser for easy viewing.

```
PowerShell 7 (x64)
PS C:\Users\sicel\OneDrive\Documents\OPS 262 - PROJECT> Get-Process | Sort-Object CPU -Descending | Select-Object Name, CPU | ConvertTo-Html -Property Name, CPU -Title "CPU Usage Report" -PreContent "<h2>CPU Usage Report</h2>" | Out-File .\CPUReport.html; Invoke-Item .\CPUReport.html
```

File C:/Users/sicel/OneDrive/Documents/OPS%20262%20-%20PROJECT/CPUReport.html	
Import favourites	Belgium Campus Po... YouTube Inbox (719) - sicelo... Dashboard _BCCo...
CPU Usage Report	
Name	CPU
Copilot	25797,34375
explorer	15728,59375
FileWatcher	6081,609375
msedge	4367,75
svchost	1496,25
ipf_helper	946,84375
OneDrive	840,1875
sihost	837,78125
msedgewebview2	814,03125
rsAppUI	717,8125
WINWORD	713,4375
rsAppUI	679,1875
ms-teams	613,171875
HPSystemEventUtilityBackground	478,28125
uihost	429,921875
ctfmon	365,546875
msedge	334,015625
rsAppUI	331,953125
msedge	300,875
...	...

References

Microsoft, 2023. Get-Process (PowerShell Documentation). [online]
Available at: <https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-process/>
Accessed: 6 August 2025

sdwheeler (2025). *Get-Process (Microsoft.PowerShell.Management) - PowerShell*. [online]
Microsoft.com. Available at: <https://learn.microsoft.com/en-gb/powershell/module/microsoft.powershell.management/get-process?view=powershell-7.5&viewFallbackFrom=powershell-6>
Accessed 8 August 2025

Belgiumcampus.ac.za. (2025). *Log in to the site | _BCCoconnect*. [online] Available at:
<https://connect.belgiumcampus.ac.za/mod/page/view.php?id=18134>
Accessed 8 August 2025

Karl-Bridge-Microsoft (2022). *Performance Counters - Win32 apps*. [online] Microsoft.com.
Available at: <https://learn.microsoft.com/en-us/windows/win32/perfctr/performance-counters-portal>.
Accessed 8 August 2025

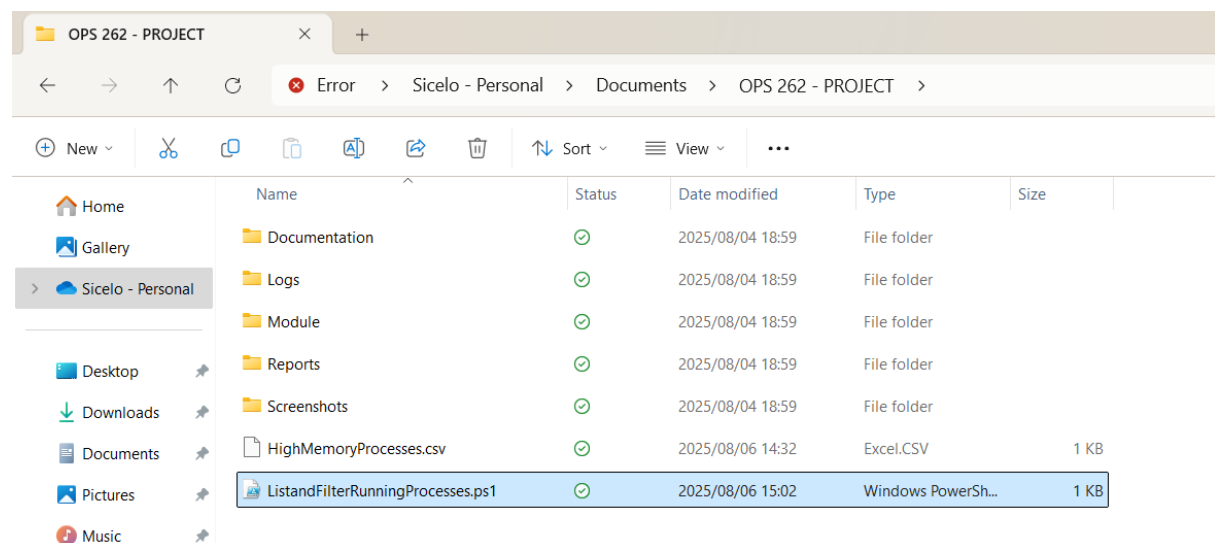
Payette, B. and Siddaway, R. (2017). *Windows PowerShell in Action*. Simon and Schuster.

Holmes, L. (2013). *Windows PowerShell cookbook : the complete guide to scripting Microsoft's command shell*. Farnham: O'reilly.

Microsoft. (2024, June 21). *Everything you wanted to know about exceptions*. Retrieved from Microsoft: <https://learn.microsoft.com/en-us/powershell/scripting/learn/deep-dives/everything-about-exceptions?view=powershell-7.5>

Appendix

Folder Structure:



The screenshot shows a Windows File Explorer window titled 'OPS 262 - PROJECT'. The address bar indicates the path: 'Error > Sicelo - Personal > Documents > OPS 262 - PROJECT >'. The left sidebar shows the navigation pane with 'Sicelo - Personal' selected. The main pane displays a table of files and folders.

Name	Status	Date modified	Type	Size
Documentation	✓	2025/08/04 18:59	File folder	
Logs	✓	2025/08/04 18:59	File folder	
Module	✓	2025/08/04 18:59	File folder	
Reports	✓	2025/08/04 18:59	File folder	
Screenshots	✓	2025/08/04 18:59	File folder	
HighMemoryProcesses.csv	✓	2025/08/06 14:32	Excel.CSV	1 KB
ListandFilterRunningProcesses.ps1	✓	2025/08/06 15:02	Windows PowerSh...	1 KB